nebio vs. Original Firmware: CPU

QB Nebulae V2 (RPi4) · nebio (**feature_newHardWareHandling**) against the original firmware (**0419df1**) · measured 2026-10-16 on the device

1. Summary

When nebio does the same work as the original firmware, **it uses less CPU**. Both were measured with the original Nebulae instrument (Csound **a_granularlooper**), updateAll() running about **400 times per second** in both, and nebio reading the hardware as often as the original does. 5 minutes each, nobody touching the module.

	Original (0419df1)	nebio, matched rates	Difference
updateAll() calls/s	400.3 (388-407)	399.9 (399.2-400.2)	same rate, nebio steadier
Whole Pi busy (4 cores = 100 %)	35.7 %	**31.8 %**	**-3.9 points (-11 %)**
Python main loop (% of one core)	71.1 %	**52.5 %**	**-18.6 (-26 %)**
CPU time per updateAll() pass	1.78 ms	**1.31 ms**	**-26 %**
I2C interrupts/s (LED writes)	5 702	**1 111**	**-81 %**
Inter-core interrupts/s	16 140	6 619	-59 %
SPI interrupts/s (ADC reads)	6 415	6 391	same (reading is matched)

- **Each updateAll() pass is 26 % cheaper**: the Python side no longer does the SPI and GPIO reads itself, it reads a snapshot that nebio has already filled.
- **The whole Pi does the same job with 11 % less CPU**, including nebio's own input thread (14.8 % of one core), which replaces the two Python encoder threads and the hardware reads in the main loop.
- **LED writes dropped by 81 %**, because nebio only writes LEDs that changed.
- **The first comparison looked worse** (section 5) because nebio then ran updateAll() about 25 % more often and scanned the hardware 2.5 times as often as the original. That was more work, not more cost per unit of work.
- **To look at:** nebio's input thread is sometimes held up for several milliseconds (worst scan up to 6.2 ms against an 833 µs period), section 4.

2. How the comparison was made

Same instrument, same work: the original sets the pace, nebio is configured to match it.

What	Original firmware (0419df1)	nebio, matched
updateAll() rate	as fast as it can: measured 400/s	at most 400/s: each snapshot waits for new ADC data (ADC 400 Hz). Measured 399.9/s
Shift register (buttons, source gate)	once per updateAll(), ~400/s	400 Hz (digital rate)
Encoder clicks	once per pass, ~400/s	400 Hz (digital rate)
ADCs (knobs, CV)	once per pass, ~400/s	400 Hz (ADC rate)
Encoders	2 Python threads, sleep(1 ms): just under 1000/s	1200 Hz (scan rate). A bit above the original, so not in nebio's favour
Gates	read once per pass	edge interrupts (no cost while no gate arrives)
Button debounce	3 equal reads	8 ms = 3 reads at 400 Hz

Setting the rates. nebio got a third rate for this (digital) next to the scan and ADC rates, a one-time C++ change. The rates are now set from Python, no recompiling:

```
# Code/nebulae/controlhandler.py
NEBIO_SCAN_HZ = 1200     # encoders, every scan (the input thread's clock)
NEBIO_DIGITAL_HZ = 400   # shift register + encoder clicks, every 3rd scan
NEBIO_ADC_HZ = 400       # knobs + CV, every 3rd scan; caps updateAll() at 400/s
nebio.set_input_rates(NEBIO_SCAN_HZ, NEBIO_DIGITAL_HZ, NEBIO_ADC_HZ)
```

Counting updateAll(). Both firmwares got the same counter in ControlHandler.updateAll(): one increment per call and a log line every 10 s (`updateAll: N calls/s`) in /tmp/nebulae_logging.log. nebio also logs its own rates (`nebio: scan 1200 Hz, snapshot 400 Hz, worst scan ... us`); its snapshot count matched the Python counter.

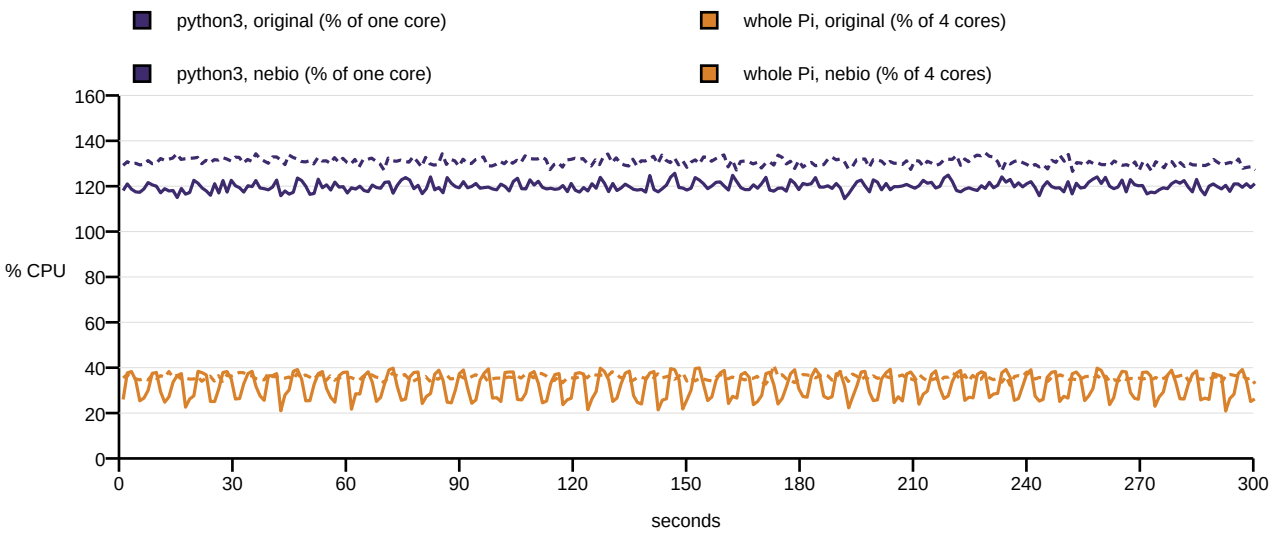
CPU. Code/scripts/cpustat.sh (sh + awk + /proc only), 300 s, one sample per second. CPU values are **% of one core** like top; the whole-Pi value uses /proc/stat, where all 4 cores = 100 %. Raspberry Pi 4, Arch Linux ARM armv7h (32 bit), kernel 5.15, governor performance, 1200 MHz. The nebio run started after 60 s of warm-up after boot.

3. Main result: Csound a_granularlooper

Program	Firmware	min %	mean %	p95 %	max %	share of Pi	max RSS
python3 (incl. Csound)	original	126.5	130.7	133.4	134.7	32.7 %	307 MB
	nebio	114.5	**119.9**	123.8	125.7	30.0 %	303 MB
whole Pi	original	32.2	35.7	37.4	41.1		
(4 cores = 100 %)	nebio	20.9	**31.8**	39.1	39.9		
temperature °C	original	41.9	43.5	44.3	45.3		
	nebio	37.5	40.5	42.4	42.8		

sclang was idle (0 %) in both runs; scsynth, jackd and pd were not running (Csound uses ALSA directly and runs inside python3).

CPU over the 5 minutes



Dashed: original. Solid: nebio. The python3 lines are steady in both runs. nebio's whole-Pi line swings between about 21 and 39 % with a period of a few seconds; this is a measurement effect, see the note below.

Note on the per-second whole-Pi values

The swing in nebio's whole-Pi line is not real load. In some seconds the whole-Pi value (e.g. 23 %) is **lower than the python3 process alone** (about 30 % of the Pi), which is impossible. The python3 process itself is as steady as in the original (standard deviation 0.5 against 0.4 points); the swing is entirely in whole Pi minus python3 (standard deviation 5.6 against 1.1 points).

Likely cause: this kernel counts busy time by sampling at 100 Hz but counts idle time exactly, and nebio's input thread wakes at 1200 Hz, an exact multiple of 100 Hz. The two drift slowly against each other, so for a few seconds the samples tend to land on busy moments, then on idle ones. The original has no fixed-rate thread like this.

The results are not affected. The 5-minute means add up the kernel's totals over about 60 such cycles, so the swing averages out (python3 30.0 % + rest 1.9 % = 31.8 %). Per-process and per-thread figures, the updateAll() rate, interrupt counts and nebio's own statistics are counted, not sampled, and show no swing. Only single

per-second whole-Pi values for nebio should not be read on their own.

Inside the python3 process

Thread	Original	nebio	What it is
main thread	71.1 %	**52.5 %**	Python main loop: updateAll(), ControlHandler, ui.py
Csound performance thread	51.8 %	49.5 %	audio; the same instrument, so about the same
encoder threads (2)	5.8 %	-	original: Python threads polling the encoders
nebio input thread	-	14.8 %	1200 Hz scan: encoders; shift register, clicks, ADCs at 400 Hz
other threads	2.0 %	3.0 %	nebio: LED writer, gate events, reset LED PWM
total	**130.7 %**	**119.9 %**	

Per updateAll() pass the main loop costs 71.1 % / 400.3 = **1.78 ms** in the original and 52.5 % / 399.9 = **1.31 ms** with nebio.

Kernel work and interrupts

	Original	nebio
user / system / idle (% of Pi)	28.9 / 6.8 / 64.3	27.0 / 5.3 / 67.7
Kernel time without a process (% of Pi)	3.0 %	1.8 %

Interrupts per second	Original	nebio	Cause
IPI3 function call	16 140	6 619	cores signalling each other
fe204000.spi	6 415	6 391	ADC reads: matched
arch_timer	3 368	2 289	kernel timer
fe804000.i2c	5 702	**1 111**	LED writes to the PCA9685
DMA IRQ x 2	94 + 94	94 + 94	audio (Csound, 512-frame blocks). Same

The extra kernel time seen in the first comparison (section 5) is gone: with matched rates the measured programs explain almost all of the Pi's load.

4. To look at: nebio's worst scan time

nebio logs the longest scan of every 10 s window. At 1200 Hz a scan has 833 µs, but the worst scan was **4.3 ms on average and 6.2 ms at most** during the run. The average rate stays exactly 1200 Hz (the thread catches up), but now and then the input thread is held up for several milliseconds.

- Likely cause: the input thread runs at **normal priority** and competes with Csound and the Python main loop.
- Effect: averages are not affected; during such a stall a fast-turned encoder could lose a step, and inputs arrive a few ms later.
- Possible fix: give the input thread real-time priority **below** the audio (jackd runs at 75), and check that the mutex shared with snapshot() cannot cause priority inversion.

5. First comparison: SC bank, nebio at its default rates

Measured earlier the same day, before the rates were matched. nebio then scanned at 1000 Hz and read the ADCs at 500 Hz, so updateAll() could run about 500 times per second against the original's ~408 (measured later with the counter):

	Original (0419df1)	nebio, default rates	Difference
updateAll() calls/s	~408 (measured later)	~500 (estimated: ADC rate cap)	about +25 %
Shift register reads/s	~408	1000	x 2.5
Whole Pi busy	26.5 %	46.0 %	+19.5 points
Python main loop	74.2 %	79.0 %	+4.8

	Original (0419df1)	nebio, default rates	Difference
nebio input thread	-	20.2 %	new
Kernel time without a process (% of Pi)	2.7 %	16.8 %	+14.1
SPI interrupts/s	5 266	7 996	+52 %
I2C interrupts/s	4 680	986	-79 %

At the time this looked like nebio being more expensive. The matched measurement shows that it was **more work**: more updateAll() passes, 2.5 times as many shift-register reads and more ADC reads. The extra kernel time came with those high rates and disappeared when they were matched. These SC numbers are kept for reference; the comparison in sections 1-3 is the valid one.

6. Hardware test (nebio)

- hwtest.py ran at boot instead of nebulae (firmware built with `create_fw_update.sh --hwtest`), log on the USB stick.
- **Inputs:** all knobs, both encoders, buttons and gates registered.
- **LEDs:** PCA9685 registers matched the expected values; ledwalk.py lit every LED and colour on its own, all worked.
- **Audio:** played during the test, 0 underruns.

7. Other findings

- The Pi is **32 bit (armv7h)**, not aarch64.
- sudo asks for the root password; pacman 7 needs `--disable-sandbox` on kernel 5.15. libgpiod 2.3.1 was installed for nebio.
- ClassLogger writes to `/tmp/nebulae_logging.log`, not to the journal.
- The Pi's clock jumps when it syncs time after boot (about 2 h on 2026-10-16); updateAll() counting uses time.time(), so one 10 s sample spans the jump. Use time.monotonic() for pacing.
- Older python2 firmwares do not run on this image (no OSC module, broken hashlib). OSC.py was copied into the Python 2 site-packages; the python3 firmwares do not need it.
- check_wifi.py and check_calibration.py use nebio through switch.py (the step 1 report says they use the old drivers).
- One boot had a USB stick drop-out (I/O errors); a_granularlooper.instr could not be copied then. Not related to nebio.

8. Next steps

Step	Why
Real-time priority for nebio's input thread (below the audio)	worst scan 4-6 ms, section 4
Measure the SC and PD banks with matched rates	only Csound is compared like for like so far
Decide the default rates (and remove the counter later)	keep the setter for tuning
Check encoder feel and knob smoothing at 400 updateAll()/s	filters work per pass; 400/s is the original's rate, so it should feel the same
Name the nebio threads (pthread_setname_np)	easier to read in top and cpustat
Optional: run with a scan rate that is not a multiple of 100 Hz (e.g. 1170/390/390)	only to confirm the cause of the per-second swing (section 3 note); does not change the results

Files

File	Content
orig_cs_granular_20261016_222919.txt / .csv	original, Csound a_granularlooper
nebio_matched_cs_granular_20261016_230314.txt / .csv	nebio 1200/400/400, Csound a_granularlooper
orig_updateall_20261016.log	original: updateAll() rate every 10 s
nebio_matched_updateall_20261016.log	nebio: updateAll() rate, nebio rates, worst scan
orig_sc_20261016_102031.txt / .csv	original, SC bank (first comparison)
nebio_sc_20261016_084140.txt / .csv	nebio default rates, SC bank (first comparison)
make_report.py	builds this PDF

All files are in measurements/cpustat/. The measurement script is Code/scripts/cpustat.sh.